

Technical Manual

How the firmware works, for someone about to change it. The design documents in `docs/design/` carry the reasoning behind each decision; this manual is the working reference that sits above them.

For building and flashing, see the Installation Manual. For operating it, see the User Manual.

Contents

1. The signal chain
2. Why duty-cycle AM needs an arcsine table
3. Tasks, cores and the real-time contract
4. Module map
5. Storage
6. HTTP API
7. Serial console
8. Memory
9. Traps that have already been paid for
10. Limits and what would lift them

1. The signal chain

There are two of these, indexed 0 and 1, and they are independent transmitters. Each has its own station, carrier frequency, modulation depth, audio low-pass, gain, output pin and power-on default. They share one Wi-Fi connection, one station list, one modulation timer and the 80 MHz LEDC clock.

```
Wi-Fi -> HTTP MP3 -> 64 kB ring buffer -> libhelix -> mono -> DC block
      -> 6-pole low-pass at 4.5 kHz -> resample to 20 kHz -> limiter
      -> lockless FIFO -> 20 kHz IRAM ISR -> arcsine table
      -> LEDC duty register -> GPIO 4 -> low-pass network -> loop    [chain 0]
                           -> GPIO 5 -> its own network  [chain 1]
```

The ring buffers live in PSRAM; everything the ISR touches stays in internal DRAM. Each chain runs its own resampler trim loop, because two stations drift against this board's crystal independently.

Each stage exists for a reason.

Ring buffer. 64 kB of raw MP3, about four seconds at 128 kbit/s. Absorbs network jitter so the decoder never starves on a slow TCP window.

Mono downmix. The carrier has one envelope. Two channels are averaged with headroom so full-scale correlated material cannot wrap.

DC block. A one-pole high-pass at 25 Hz. MP3 decoders emit a DC offset on some material, and DC here shifts the carrier bias rather than being inaudible.

Low-pass. Three cascaded biquads, a 6th-order Butterworth at 4.5 kHz. It is 41 dB down at 10 kHz where a 2nd-order design manages 14 dB. This is what keeps the transmission inside a 9 kHz channel.

Resampler. Linear interpolation from the stream rate to 20 kHz. It also nudges its own step to hold the sample FIFO near half full, absorbing the drift between the station's clock and this board's crystal. Without that the FIFO walks to one end and clicks every few minutes. The correction is bounded at 5000 ppm, far more than two crystals can differ and small enough to be inaudible.

AGC. Between the fixed gain and the limiter. Tracks programme RMS and drives gain towards a target, because stations differ by more than 10 dB in average level and a peak limiter cannot close that gap: it only pushes peaks down. Asymmetric slew, 200 ms down and 3 s up, bounded -6/+18 dB, gated at -50 dBFS so speech pauses do not lift the noise floor. See [EH-11](#).

Limiter. 1 ms attack, 250 ms release, ceiling at 0.95 of full scale. Last in the chain, as the safety net: the AGC targets an average, and peaks are not its job.

FIFO. 2048 samples, 102 ms at 20 kHz. Single producer, single consumer, lockless. The decode task writes the head, the ISR writes the tail, and neither touches the other's index. This is why the decode task and the ISR must share a core.

2. Why duty-cycle AM needs an arcsine table

This is the part of the design worth understanding before changing anything.

The amplitude a square wave radiates at its own frequency goes as $\sin(\pi D)$, where D is the duty cycle. That function **peaks** at 50% duty, so its slope there is zero. Modulating duty about 50%, which is the obvious reading of the brief, produces almost no amplitude modulation. Swinging from 25% to 75% moves the radiated amplitude from 0.707 to 0.707.

So the firmware puts the modulation **peak** at 50% duty, sits the unmodulated carrier at $1/(1+m)$ of it, and inverts the sine into a 512-entry table built once at configuration time:

$$D = \text{asin}(a) / \pi$$

The interrupt is then a lookup, an add, a clamp and a register write. The envelope comes out linear in the sample.

For the default depth of 0.70 the quiescent duty is $\text{asin}(1/1.70)/\pi = 0.2002$, which lands on 51 of 256 register steps. Seeing `quiescent duty 51/256` in the boot log is confirmation the table built correctly.

This is `KI-01` and `LL-02`. The host test suite asserts it, including a check that the brief's original 50%-centred scheme really is flat.

Noise shaping

At 200 kHz on the 80 MHz LEDC clock there are 400 clocks per period, so the duty register has 8 usable bits. Around the bias point that leaves roughly 6.5 bits of amplitude resolution. First-order error feedback on the duty quantiser carries the rounding error into the next tick, which pushes most of that noise above the 4.5 kHz audio band. Two adds in the ISR. Set

`RF_NOISE_SHAPING` to 0 to compare on a scope.

None of this has been measured on hardware.

3. Tasks, cores and the real-time contract

Task	Core	Priority	Stack	Count
Modulation ISR	1	interrupt	n/a	1, serving every chain
Decode	1	10	12288	one per chain
Stream	0	5	8192	one per chain
Wi-Fi, lwIP, httpd	0	IDF defaults	IDF defaults	1
Console REPL	0	IDF default	4096	1

Measured with two chains playing different stations **on an ESP32-S3R8, the module with 8 MB octal PSRAM**: each decoder sustains 42.1 frames/s, which is the correct rate for 48 kHz MP3, with modulator underruns static and both FIFOs near full. Core 1 carries two decoders and the 20 kHz ISR without strain.

A quad-PSRAM module does it too. Measured on a Waveshare ESP32-S3-Zero (2 MB quad PSRAM) on 2026-09-18: two 48 kHz 128 kbit/s streams at once, 41.8 and 41.8 frames/s, zero underruns, on both the pre-refactor build and the refactored one. The same board had been seen running 21% slow on both chains shortly after a flash, which is `KI-14`: unexplained, not reproducible since, and left open rather than explained away.

The ISR's own cost has never been measured directly, and `WI-1`'s decode-cost measurement is still outstanding. **Know which module you have** before trusting either set of numbers.

The split is not arbitrary. Core 0 carries everything that can block on the network. Core 1 carries the audio path and nothing else, so no decode ever waits on a socket.

Three rules hold the real-time behaviour together.

The modulation ISR must not touch flash. It is `IRAM_ATTR`, its tables are `DRAM_ATTR`, and `sdkconfig.defaults` sets `CONFIG_GPTIMER_ISR_IRAM_SAFE`, `CONFIG_GPTIMER_CTRL_FUNC_IN_IRAM` and `CONFIG_GPTIMER_ISR_HANDLER_IN_IRAM`. Without those the ISR lives in flash and the first cache miss during an SPI flash operation takes the carrier with it.

The ISR must be registered from core 1. A gptimer ISR is allocated on the core that registers it, so `app_init()` runs from a task pinned to `TASK_CORE_AUDIO`, not from `app_main()`. See `main.c`. Every decode task is pinned there too, for the same FIFO reason.

Stop the ISR across a flash write. Being IRAM-safe is necessary but not sufficient. Taking a 20 kHz interrupt continuously through a cache-disabled window starves the interrupt watchdog and the board resets before the write lands. `carrier_suspend()` and `carrier_resume()` bracket every `nvs_commit()`. This is `KI-07` and `Â§9` covers what it cost to find.

4. Module map

File	Responsibility
<code>main.c</code>	Bring-up order, and nothing else
<code>app.c</code>	Glue: what plays, what the RF settings are
<code>state_doc.c</code>	The state document, for <code>/api/state</code> and the console's <code>state</code>
<code>audio/decoder.c</code>	libhelix, the DSP chain, the test tone
<code>audio/dsp.c</code>	Filters, resampler, limiter, duty table. Host-testable
<code>audio/sample_fifo.h</code>	The lockless FIFO
<code>net/stream.c</code>	HTTP fetch into the ring buffer, retry and backoff
<code>net/wifi_conn.c</code>	Joining, the setup portal, the captive DNS, the heartbeat
<code>net/discover.c</code>	Radio-Browser search
<code>net/webui.c</code>	HTTP server and the JSON API
<code>net/www_page.h</code>	The whole UI. Assembles the three below, and what is served is their concatenation byte for byte
<code>net/www_style.h</code> , <code>www_body.h</code> , <code>www_script.h</code>	The stylesheet, the markup, the script
<code>rf/carrier.c</code>	LEDC, the gptimer, the modulation ISR
<code>button.c</code>	The BOOT button, and the factory reset a five-second hold triggers
<code>cli/console.c</code>	The REPL, and the one loop that registers every command
<code>cli/cli_reply.c</code>	The <code>ok</code> / <code>error:</code> convention and the shared argument parsers
<code>cli/cmd_radio.c</code>	play, stop, station, rf, tone, discover
<code>cli/cmd_net.c</code>	wifi, hostname, web
<code>cli/cmd_system.c</code>	status, state, log, reboot
<code>store/settings.c</code>	Mounting NVS and the load order. <code>settings.h</code> is the one public header
<code>store/settings_stations.c</code>	The <code>radio</code> namespace: the station list and the indices into it

File	Responsibility
store/settings_wifi.c	The <code>wifi</code> namespace: saved credentials
store/settings_rf.c	The <code>rf</code> namespace: per-transmitter carrier and audio settings
store/settings_sys.c	The <code>sys</code> namespace: hostname, and whether the web runs
store/settings_internal.h	What those four share, including the carrier bracket every NVS write needs (KI-07)

Two seams worth knowing before you change anything here. The command groups under `cli/` each publish a table and `console.c` registers all of them in one loop, so a command lives with its neighbours rather than in one long file; what any of them prints is a contract (design 07 Â§2) that the programmer tool parses, so moving a command is free and rewording its reply is not. And `store/` is split one file per NVS namespace, matching the schema in design 04 Â§4: each owns its own RAM cache, its own save path and its own defaults, and none of them reads another's.

`audio/dsp.c` is deliberately free of ESP-IDF headers so it compiles on a host. `test/host/` builds it with the test suite. If a change to `dsp.c` breaks that build, the change is at fault, not the test.

5. Storage

Four NVS namespaces, one file each under `store/`. Keys are 15 characters or fewer, as NVS requires. Design 04 holds the schema table, and a new key is a change to that table in the same commit.

Namespace	Keys	Holds
radio	<code>cnt</code> , <code>last</code> , <code>def</code> , <code>na0..n</code> , <code>ur0..n</code> , <code>tr0..n</code>	Station list, last played, power-on default, per-station trim
wifi	<code>n</code> , <code>s0..n</code> , <code>p0..n</code>	Credentials, newest first
rf	<code>on</code> , <code>hz</code> , <code>depth</code> , <code>lpf</code> , <code>gain</code> , <code>lvl</code> , <code>agc</code> , <code>agct</code> , <code>fdm</code> , <code>fdr</code> , <code>fdd</code> , and the same suffixed <code>1</code> for the second transmitter	Transmitter settings
sys	<code>host</code> , <code>web</code>	The hostname, and whether the web interface runs

Stations and credentials are cached in RAM and written through, so a web handler never blocks on flash while holding anything.

Indices name a station, not a position

`last` and `def` are both indices into the station list. Any operation that renumbers the list has to carry them along, or reordering silently changes which station the board wakes up on.

`settings_station_move()` remaps both through `remap_after_move()`. `settings_station_delete()` decrements them past the gap, and clears `def` outright if the starred station is the one deleted, because a default nobody chose is worse than none.

`def` is -1 when unset, in which case boot falls back to `last`.

6. HTTP API

Everything is JSON. Design 04 is the contract.

Method	Path	Body	Returns
GET	/		The page
GET	/api/state		Everything. Polled once a second
GET	/api/wifi/scan		[{ssid, rssi, secure}]. Blocking
GET	/api/discover	?q= &by=name tag country	[{name, url, country, codec, bitrate, votes}] or {error}
POST	/api/station/play	{index} or {url}	{ok}
POST	/api/station/add	{name, url}	{ok}
POST	/api/station/delete	{index}	{ok}
POST	/api/station/move	{from, to}	{ok}
POST	/api/station/default	{index}, negative to clear	{ok}
POST	/api/stop	{}	{ok}
POST	/api/rf	{on, hz, depth, lpf, gain}	{ok}
POST	/api/tone	{on, hz}	{ok}
POST	/api/wifi/add	{ssid, pass}	{ok}, then reboots if from the portal
POST	/api/reboot	{}	{ok}
POST	/api/hostname	{host}	{ok}
POST	/api/web	{on: false}	{ok}, then the server stops

Every POST answers `{"ok":bool}` and, on failure, `{"error":"ESP_ERR_..."}`. The page shows that string to the operator rather than swallowing it.

`station/move` performs a splice, not a swap, which is exactly drag-and-drop semantics. The UI reuses it for both the arrows and the drag.

A 404 redirects to `/`, which is what makes the captive portal open by itself. Captive-portal probes are frequent enough that the `httpd_uri` log tag is turned down to `ERROR`, or their warnings bury everything else.

7. Serial console

An `esp_console` REPL on the USB Serial JTAG peripheral, started before `wifi_conn_start()`, which blocks, so the console answers while join attempts are still running.

`cli/console.c` holds the REPL and the registration loop; the commands themselves are grouped in `cmd_radio.c`, `cmd_net.c` and `cmd_system.c`, and the reply convention and the argument parsers are in `cli_reply.c`. The grouping is for the reader: `esp_console` sorts its own command list, so nothing a program sees depends on which file a command lives in.

It reaches every function the page does, and design 07 Â§2 is its reference. Commands added with it end with one `ok` or `error:` line and return 0, because a non-zero return makes the REPL print a line of its own after the reply. `discover` runs its search on a 6 kB task created for the search, because the TLS handshake does not fit the REPL's 4 kB stack and a bigger stack would hold internal RAM permanently (LL-07).

`log off` sets the global log level to `NONE` so `status` can be polled by a script. `log on` restores `INFO` and re-applies the `httpd_uri` suppression. The quiet mode is not persisted: logging is on after every boot.

`status` prints one line, fixed field order:

```
state=connected ssid="..." ip=... rssi=-55 drops=0 saved=1 heap=... heapint=...
heapintmax=... uptime=134 carrier0=on hz0=200000 depth0=70 playing0=1
underruns0=0 carrier1=on ... host=amtx web=on
```

(One line on the board; wrapped here.)

8. Memory

Roughly 8 MB of heap free at rest. **That number is close to meaningless**, and watching it instead of internal RAM cost a debugging session (LL-07).

The figure that constrains this board is internal DRAM, and there is about 19 kB of it free with two chains playing. Everything scarce competes for it: Wi-Fi buffers, DMA, task stacks and mbedTLS. Both the Diagnostics page and the CLI `status` line report internal free and its largest free block for exactly this reason.

Consumer	Size	Where
Stream ring buffer	64 kB each, 128 kB total	PSRAM
Decoder working buffers	15 kB each, 30 kB total	PSRAM
Sample FIFO	4 kB each, 8 kB total	Internal, and must stay there
Duty table	1 kB each	Internal, and must stay there
Discovery reply buffer	20 kB, transient	PSRAM
mbedTLS session	Tens of kB, transient	PSRAM, via <code>CONFIG_MBEDTLS_EXTERNAL_MEM_ALLOC</code>
PSRAM DMA reserve	32 kB	Internal, reserved by IDF

The two rows marked "must stay there" are the modulation ISR's, and PSRAM is reached over SPI. Nothing on that path may live in it.

PSRAM is **on**. The module is an **ESP32-S3R8 with 8 MB octal PSRAM**, which answers Design 06 Â§2.1, and `sdkconfig.defaults` sets `CONFIG_SPIRAM_MODE_OCT`. The stream ring buffer moves there by itself, because `stream_init()` asks for PSRAM first and falls back.

Nothing on the real-time path lives in PSRAM. It is reached over SPI, so it must never be in the modulation ISR's way: the ISR stays `IRAM_ATTR` and its tables `DRAM_ATTR`, exactly as before.

`CONFIG_SPIRAM_IGNORE_NOTFOUND` keeps a board without PSRAM booting rather than failing at startup, so the default stays safe on other modules.

The tight spot is a TLS handshake. mbedTLS with the full certificate bundle takes the largest contiguous block while it runs, so an allocation attempted after the handshake can fail with 99 kB apparently free. `discover.c` claims its reply buffer before opening the connection for exactly this reason.

9. Traps that have already been paid for

Read this section before spending a day on any of them again.

KI-07 : an NVS write with the modulator running resets the board. See Â§3. It presented as a Wi-Fi fault for a whole session, because the setup portal is where you first try to save something. `settings_set_last_station()` hid it further by returning early when the index has not changed, so an ordinary boot performs no real flash write and never trips it.

LL-04 : `max_redirection_count` does nothing on the `open()` path.

`esp_http_client` follows redirects only inside `esp_http_client_perform()`. A stream cannot use `perform()`, so it uses `open()` and `fetch_headers()`, and the config field is never consulted. Follow them by hand: close, then `esp_http_client_set_redirection()`, then open again.

KI-05 : Shoutcast v1 answers `ICY 200 OK`. That is not a status line

`esp_http_client` can parse. Presenting a browser User-Agent gets a normal response.

TLS was compiled in all along. The UI carried a "TLS is not compiled in" warning for a long time. It was wrong. `CONFIG_ESP_HTTP_CLIENT_ENABLE_HTTPS` and the full certificate bundle were both set. What was missing was `crt_bundle_attach` on the stream client, which is a different problem with the same symptom, and the wrong note discouraged anyone from looking.

The console goes to the native USB port. `CONFIG_ESP_CONSOLE_USB_SERIAL_JTAG` means a USB-to-serial bridge on the same board shows you the ROM banner and nothing more. A panic would be equally invisible there.

10. Limits and what would lift them

Limit	Cause	What would lift it
MP3 only	libhelix is the only decoder	EH-04
10 discovery results	Internal heap during a TLS handshake. PSRAM is now on, so this could be raised and has not been retested	Raise DISCOVER_RESULTS_MAX
20 stations	STATION_MAX	Raise it. NVS has room
Carrier below roughly 300 kHz	8-bit duty on an 80 MHz LEDC clock	Fewer duty bits, at a cost in resolution
6.5 bits of amplitude resolution	Same	EH-05 , a faster modulation rate
No stream metadata	ICY is interleaved and not stripped	EH-08
Fading is a table rebuild	Level is baked into the duty table	Index the table by amplitude instead, making level a runtime multiply
No authentication	Not implemented	KI-06
Harmonics across medium wave	A square wave carrier	An output network. KI-02 . Not fixable in firmware

What has never been verified

The firmware runs. The audio chain is tested on a host and observed working on hardware, from an internet stream through to the duty register.

Nothing has been measured off-air. There has been no receiver, no scope and no spectrum analyser on this project at any point. The RF behaviour, the harmonic content, the modulation quality and the actual radiated field are all unobserved. Design 06 lists what else was assumed.